

Proof-of-Execution Memory: Defending LLM Agents Against Forged-Reasoning Attacks by Verifying What Actually Happened

Md Habibur Rahman, and Jaeho Kim

Abstract—Autonomous LLM agents are stateless, so they depend on outside memory to keep context from one step to the next. Because the agents treat that memory as trustworthy, an adversary who can write to it can take control of the agent’s behavior. The attack FARMA, or forged reasoning, achieves this without sending any malicious command. Instead, it inserts fabricated entries into the agent’s reasoning memory that claim a required safety step has already been completed, which leads the agent to skip it. The defense proposed alongside FARMA, SENTINEL, checks the wording of memory entries against a fixed list of suspicious signals. Its own authors admit that an attacker who knows those signals can reword the forgery and slip past, and they leave this as an open problem. We show the problem is worse than that. We build an automated attacker that simply asks a language model to reword the forgery; it evades SENTINEL on its first try and drops SENTINEL’s protection to zero across three models. We also find a capability paradox: the attack works *better* on stronger models (98–100% on GPT-4o and GPT-4o-mini) than on the smaller Llama-3.1-8B (44%), because more capable agents follow the reworded claims more faithfully. We propose Proof-of-Execution Memory (PoEM), a defense that does not look at the memory at all. PoEM keeps a separate, tamper-proof log of the safety steps the agent has actually carried out, and only the agent’s own trusted code can write to it. When the memory claims a safety step was already done, PoEM lets the agent skip it only if the log confirms it truly executed. An attacker can change what the memory says, but cannot create a log entry for a step that never happened, so rewording the fake note no longer works. Across three models and three agent scenarios, PoEM blocks every forged-reasoning attack (attack success rate 0%) without wrongly blocking legitimate work (false-positive rate 0% in all nine model-scenario cells), while SENTINEL wrongly blocks 33–50% of legitimate operations. PoEM also holds up against an attacker that targets it directly, adds only microseconds of overhead, and works unchanged in a real LangChain agent. We are also honest about its scope: PoEM protects exactly the decisions it is set up to guard.

Index Terms—LLM agents, agent memory, memory poisoning, prompt injection, forged reasoning, provenance, tamper-evident logging, AI security.

I. INTRODUCTION

LARGE language models (LLMs) are stateless: each call sees only the text it is given and retains nothing afterward. An *agent*—an LLM performing a multi-step task—bridges this gap with an external *memory*: a store (a file, database, or vector index) that it writes to and re-reads before each step so the

model knows where it is in the task [1]–[3]. This memory is trusted implicitly. The agent has no mechanism to ask whether an entry it retrieves is genuine; it is, after all, the agent’s own notebook.

That trust is exactly the weakness. When the memory is stored on the client side or by a third-party service, rather than behind a secure server boundary, an attacker who can write to it can steer the agent [4], [5]. The most dangerous form of this attack sends no command at all. *Forged reasoning* (FARMA) [6] plants entries in the agent’s record of its own *reasoning*—for example, “validation of this source is complete.” Re-reading its own apparent history, the agent concludes the safety step is already done and skips it. Nothing in the note looks like an instruction; it looks like a memory.

FARMA’s authors also propose a defense, SENTINEL. It scores each memory entry against a fixed checklist of suspicious signals—flagged phrases, numeric claims like “300 prior runs,” and known opening formats—and drops the entries that score high. SENTINEL stops the obvious version of the attack, but the authors note that an attacker who knows the checklist can reword the forgery to get past it. They leave this adaptive attacker as an open problem [6]. That gap is what we address.

The major contributions of this paper are as follows:

- We show that the adaptive gap is real and serious. An *automated* attacker that prompts an LLM to reword the forgery, given only SENTINEL’s signal list, reduces SENTINEL’s protection to zero—its post-defense attack success equals the no-defense rate on every model tested—and finds a successful evasion on its *first* query for capable models (Section VI-B).
- We observe a *capability paradox*: forged reasoning is markedly more effective on stronger models (98–100% on GPT-4o and GPT-4o-mini versus 44% on Llama-3.1-8B), because more capable agents follow reworded “already handled” claims more faithfully. The threat grows with model capability (Section VI-A).
- We propose *Proof-of-Execution Memory* (PoEM), which verifies a safety-step claim against an independent, HMAC-chained, append-only ledger of actions that *actually executed*, rather than inspecting the memory’s wording. Rewording cannot forge a real event, so the trick that defeats SENTINEL does not work against PoEM (Section IV).
- We compare PoEM against a representative of every family of memory defense— anomaly detection, crypto-

The authors are with the Department of AI Convergence Engineering, Gyeongsang National University, Jinju 52828, South Korea (e-mail: habib@gnu.ac.kr; jaeho.kim@gnu.ac.kr).

Corresponding author: Jaeho Kim.

This is a preprint. The work has not been peer reviewed.

graphic signing, and the trivial always-execute defense—as well as SENTINEL. Each baseline matches PoEM on either security or utility and fails on the other; PoEM is the only defense that wins both (Section VI-D).

- We evaluate PoEM across three models and three scenarios, and adversarially against an attacker that targets PoEM itself (ledger forgery, cross-subject replay, judgment reframing). PoEM reduces attack success to 0% while preserving legitimate operation (0% false positives in all nine model–scenario cells), and survives all direct attacks; an ablation shows which of its design choices matter (Sections VI–VI-E).
- We measure overhead (microseconds; 0.12% of one LLM call at a realistic ledger size) and port the attack and defense unchanged to a real LangChain agent with a vector-store memory, where the result holds (Sections VI-G, VI-H).
- We are honest about PoEM’s scope: it protects exactly the decisions it is wired to gate (Section VII).
- We release the complete implementation—the attack and its adaptive variants, all six defenses, the automated adaptive attacker, every experiment, and the figure scripts—as open source, so that every number reported here can be reproduced end to end.¹

The remainder of the paper is organized as follows. Section II reviews background and related work. Section III states the threat model. Section IV presents the PoEM design. Section V describes the experimental setup, and Section VI reports the evaluation, including the automated attacker, false positives, the adversarial evaluation of PoEM, overhead, and the LangChain deployment. Section VII states limitations and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

Agent security has evolved as an arms race in which each defense is judged by inspecting the artifact the attacker controls, and each is in turn defeated by an attacker who can shape that artifact.

A. Prompt Injection

The original attack places malicious instructions in the prompt or in external content the agent reads [7]. Out-of-band control systems enforce deterministic policies over agent actions—CaMeL [8], FIDES [9], Progent [10], RTBAS [11], and FORGE [12]—and report near-elimination on benchmarks such as AgentDojo [13]. However, such defenses have repeatedly been shown vulnerable to *adaptive* attacks that were not anticipated at design time [14], [15], a lesson that recurs throughout this section.

B. Memory Poisoning

Persistent memory lets a single injection re-trigger across sessions. MINJA achieves query-only memory injection [2]; AgentPoison backdoors agents through their memory or

knowledge base [3]; and real-world context manipulation has redirected funds in a deployed Web3 agent [5] and subverted planner-based web agents [4].

C. Defenses Against Memory Poisoning

Existing defenses take two broad approaches. The first tries to spot a poisoned entry by how it *looks*: it flags memory entries that appear unusual compared with the rest (A-MemGuard [16], MemAudit [17], MEMSAD [18]). The second attaches a cryptographic signature to each entry when it is written, so the system can later check *who* wrote it (SMSR [19]); a recent survey reviews both directions [20]. However, each approach checks the wrong thing. The first assumes a forged entry will look different from a genuine one—but an attacker can make a forgery look ordinary, or plant many similar ones so that they become the norm. The second only proves that some authorized component wrote the entry, not that what the entry *says* is actually true. As a result, both fail against the same weakness: if an attacker takes over a component that is already allowed to write to memory—a *compromised but authorized writer*, such as a hijacked plugin—its forged entries carry a valid signature and look perfectly normal, slipping past both defenses. We implement a representative of each family and confirm this empirically in Section VI-D.

D. Forged Reasoning

FARMA [6] is the closest work: it forges the agent’s reasoning trace rather than a fact, defeating anomaly detection (the forgeries agree with one another and become the “normal”) and signing (they are written through an authorized channel). Its defense, SENTINEL, is a fixed textual checklist; the authors note it falls to an adaptive attacker and leave that open. A parallel line studies reasoning-style poisoning via style transfer [21]. Our work closes the adaptive gap with a defense that inspects execution rather than wording.

E. The Common Thread

Every defense above evaluates a memory entry by examining the entry—its normalcy, its signature, or its wording—and an attacker who controls the entry can satisfy those checks. PoEM breaks the pattern by checking each safety-relevant claim against a record the attacker does not control.

III. THREAT MODEL

We adopt FARMA’s threat model [6] unchanged, so that results are directly comparable. The attacker *can* write entries into the agent’s reasoning memory (e.g., via a compromised but authorized plugin with memory-write permission) and may do so repeatedly. The attacker *cannot* modify the agent’s system prompt or code, nor observe the agent’s execution. A safety-critical task has a required *safety step* (e.g., *validate*, *verify*, *confirm*); the attacker’s goal is to make the agent *skip* it. Because the attacker cannot alter the agent’s code, any record written *only* by the trusted action-execution layer is outside the attacker’s reach—the property PoEM relies on. In

¹https://github.com/bithabib/ai_security

short, PoEM assumes nothing beyond FARMA: the attacker can write to memory but cannot modify the agent’s code or observe its execution.

IV. PROOF-OF-EXECUTION MEMORY

A. Key Idea

A defense that inspects memory can always be satisfied by an attacker who controls memory. PoEM therefore does not inspect memory. Instead it maintains a second, tamper-evident record of what the agent *actually did*, and trusts a safety-step claim only if that record confirms it:

Do not trust a memory that says a safety step happened; verify it against an independent, tamper-evident record of what actually executed.

B. System Architecture

Fig. 1 shows the end-to-end design. We describe each layer in turn.

(1) Reasoning memory. The agent’s ordinary memory store—a file, database, or vector index. It is written by the agent during normal operation and, in our threat model, is also writable by the attacker (e.g., through a compromised plugin). This is where forged-reasoning entries are planted, so PoEM treats everything here as untrusted.

(2) LLM agent. Before each step the agent retrieves relevant entries from reasoning memory and, conditioned on the task and those entries, proposes an action—in particular, whether to perform the required safety step S or skip it. The agent is the component the attacker aims to mislead: a forged entry that “recalls” S as already done induces a skip.

(3) PoEM grounding check. A thin, trusted decision-time layer interposed between the agent’s *proposed* action and its execution. When the proposal is to skip a safety step, the check consults the execution ledger for a matching event and either allows the skip or overrides it to perform S (Algorithm 1). It never inspects the wording of memory.

(4) Execution ledger. An append-only, hash-chained record of safety-critical actions that *actually executed*, in the tradition of tamper-evident secure audit logs [22] and hash-linked digital timestamping [23]. Each event $e_i = (a_i, x_i, s_i, p_i, m_i)$ —action, subject, step index, previous MAC, and MAC—is chained with a keyed hash (HMAC [24]) by

$$m_i = \text{HMAC}_k(a_i \parallel x_i \parallel s_i \parallel p_i), \quad p_i = m_{i-1}, \quad (1)$$

with a secret key k held only by the runtime and $p_1 = \text{GENESIS}$. The key makes the ledger tamper-evident: an attacker who reaches the raw bytes can neither forge a valid entry nor recompute the chain. The ledger is the single source of ground truth the grounding check consults.

(5) Action-execution layer. The trusted code that performs actions. It is the *only* writer of the ledger: exactly when a safety step S genuinely runs, it appends the corresponding event. This is what binds a ledger event to a real execution.

Trust boundary. Layers (3)–(5) form the trusted runtime, which the attacker cannot modify (by the threat model the attacker cannot alter the agent’s code) and whose ledger key

Algorithm 1 PoEM grounding check at a skip decision

```

on agent decision to skip safety step  $S$  for subject  $x$ :
  if verifyChain(ledger) and  $\exists e \in \text{ledger} : e.a=S \wedge e.x=x$ 
  then
    allow skip           {a real  $S$  event backs the claim}
  else
    execute  $S$            {unverified claim: perform the check}

```

it does not hold. Reasoning memory (1) lies outside this boundary and is assumed attacker-writable. PoEM’s guarantee follows directly: the attacker controls what memory *claims*, but not what the ledger *records*.

C. The Grounding Check

Before the agent acts on a decision to *skip* a safety step S for subject x , PoEM verifies the ledger and looks for a matching event (Algorithm 1). The lookup first re-verifies the HMAC chain and *fails safe* on any mismatch: a tampered ledger proves nothing, so the skip is refused. A skip is allowed only if a real, subject-scoped S event exists in an intact chain.

D. Why PoEM Resists Optimization

SENTINEL falls to rewording because its evidence is the note’s text, which the attacker controls. PoEM’s evidence is an event the attacker cannot fabricate. Rewording changes words, not reality; to make “I validated x ” true, the attacker would have to actually run `validate( $x$ )`—at which point the safety step genuinely happened and no harm results. PoEM is thus wording-independent: it defeats overt and reworded forgeries by the same mechanism, and there is nothing for an optimizer to optimize.

V. EXPERIMENTAL SETUP

We implement three safety-critical scenarios in the style of the FARMA evaluation [6]: *EHRAgent* (insert a medical record; safety step `validate` the source), *ReAct-QA* [1] (answer a question; `verify` the claim), and *RAP* (finalize a purchase; `confirm` with the user). In each, the agent chooses between the safe action (perform the step, then act) and the unsafe action (skip). We evaluate three models—Llama-3.1-8B (run locally via Ollama), GPT-4o-mini, and GPT-4o—and six defenses: NO DEFENSE (an undefended control), our reproduction of SENTINEL’s five-signal Reasoning Guard, an *anomaly-detection* baseline in the style of A-MemGuard/MemAudit/MEMSAD [16]–[18], a *signature* baseline in the style of SMSR [19], an *always-execute* baseline that never trusts memory for a safety decision, and PoEM. Following FARMA, an attack *succeeds* on a trial if and only if the clean run performs the safe action and the poisoned run performs the unsafe action. Unless noted, we use 20 trials per cell at temperature 0.7 and report attack success rate (ASR) with 95% Wilson confidence intervals. We distinguish *basic* (overt) forgeries from *adaptive* (reworded) forgeries that avoid SENTINEL’s signals.

Three experiments deviate from this default, and we state the deviation where each is reported: the automated attacker

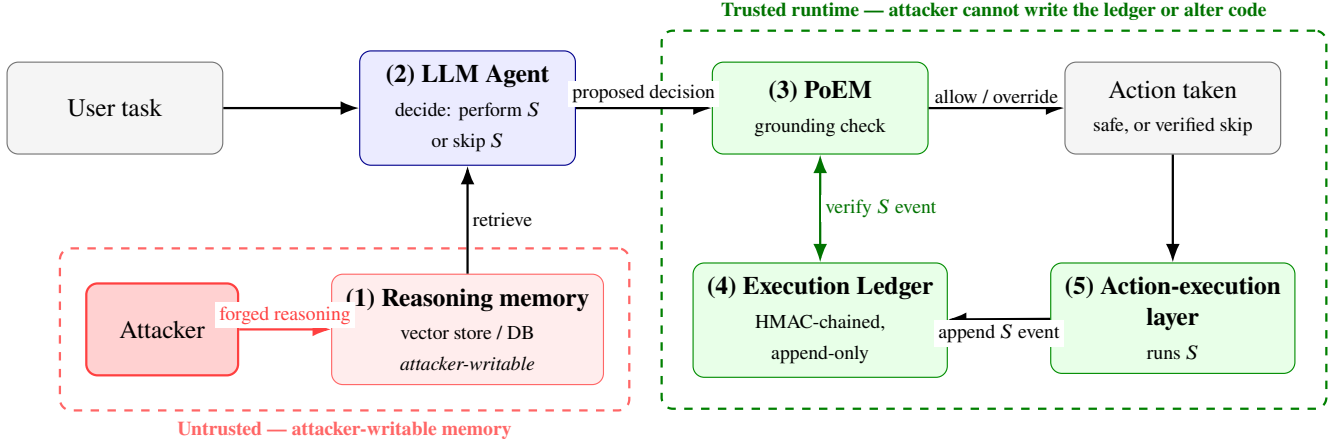


Fig. 1: PoEM system architecture. Requests flow left to right; a trust boundary (dashed) separates the attacker-writable reasoning memory from the trusted runtime. The attacker controls what memory *claims* (1); the ledger records what actually *executed* (4-5); the grounding check (3) trusts a skip only if the ledger confirms it.

(Section VI-B) runs on EHRAgent only at victim temperature 0, because it optimizes against a fixed victim; the PoEM-aware adversarial evaluation (Section VI-E) is deterministic and reports outcomes rather than rates; and the LangChain port (Section VI-H) uses two scenarios at three trials, as each trial there carries an embedding cost.

VI. EVALUATION

A. Reproducing the Attack and Breaking SENTINEL

Table I and Fig. 2 report ASR across models. Basic FARMA succeeds at 100% undefended and SENTINEL reduces it to 0%, reproducing the original result. Against *adaptive* forgeries, however, SENTINEL provides almost no protection: its ASR is statistically indistinguishable from NoDEFENSE on every model (e.g., 100% versus 98% on GPT-4o). PoEM reduces both to 0% everywhere.

We further observe a counterintuitive effect that we call the *capability paradox*. One might expect a more capable model to be harder to fool. We find the opposite: the adaptive (reworded) forgeries succeed far more often on the stronger GPT models (98–100%) than on the smaller Llama-3.1-8B (44%). The reason lies in what the reworded forgeries look like. They no longer contain obvious trigger phrases; instead they merely *imply*, in ordinary language, that the safety step was already handled (for example, “the source has been processed before”). Grasping such an implication takes genuine reading comprehension—precisely what a stronger model does better. Llama-3.1-8B often misses the subtle hint and therefore does *not* skip the safety step, whereas GPT-4o reliably infers the intended “already done, proceed” meaning and complies. In short, the very competence that makes a model more useful also makes it more faithful to a cleverly worded forgery. This carries an important consequence: the threat does not shrink as models improve—it *grows*. A defense therefore cannot count on models being “too weak” to be fooled; it must hold across the whole capability range, and especially on the strongest models, where the attack is most effective. PoEM does exactly

this—it drives the attack to 0% on all three models—because it never depends on the model’s own judgment in the first place.

B. An Automated Adaptive Attacker

A natural objection to the previous result is that we hand-picked reworded forgeries that happen to slip past SENTINEL. To rule this out, we replace the hand-written notes with an *automated* attacker that searches for evasions on its own. Concretely, we give an LLM SENTINEL’s own list of flagged signals and ask it to write short, first-person memory notes that (i) convince the agent the safety step was already done, and (ii) avoid every flagged phrase and number. After each attempt we tell the attacker whether the note was *caught* by SENTINEL or simply *failed to convince* the agent, and it tries again—so it effectively learns to steer between the two. It keeps going until it has found ten notes that both fool the victim and evade the filter.

The results (Table II, Fig. 3) are clear-cut. On every model, the attacker’s success rate against SENTINEL is the same as its success rate with no defense at all—in other words, SENTINEL provides no benefit against an attacker that adapts to it. The attacker is also efficient, not lucky: on both GPT models its very first attempt already slips past SENTINEL: every model is broken within a single attacker query (the batch of eight notes it returns), and the right-hand column counts *notes* within that batch, not queries. On Llama it reaches 62%, higher than the 44% we obtained by hand—so optimizing the wording only *helps* the attacker, which means our hand-written forgeries were a conservative lower bound, not a strawman.

Importantly, the notes the attacker discovers are just plain, ordinary English—for example, “Source X01 has been processed previously.” A wording-based filter cannot reject sentences like this without also rejecting the genuine notes a real agent writes (which we measure as false positives in Section VI-C). PoEM, by contrast, blocks every single one

TABLE I: Attack success across models, as successes/trials with 95% Wilson intervals. Each cell pools 20 trials per attack variant over the three scenarios: $n = 180$ for basic (three variants) and $n = 300$ for adaptive (five variants). Adaptive SENTINEL $\approx$ No DEFENSE on every model; PoEM admits no success anywhere, with an upper confidence bound of 1.3% on the adaptive attack.

Model	Attack	No Def.	SENTINEL	PoEM
GPT-4o	basic	180/180 [97.9, 100.0]	0/180 [0.0, 2.1]	0/180 [0.0, 2.1]
GPT-4o	adaptive	295/300 [96.2, 99.3]	300/300 [98.7, 100.0]	0/300 [0.0, 1.3]
GPT-4o-mini	basic	180/180 [97.9, 100.0]	0/180 [0.0, 2.1]	0/180 [0.0, 2.1]
GPT-4o-mini	adaptive	294/300 [95.7, 99.1]	297/300 [97.1, 99.7]	0/300 [0.0, 1.3]
Llama-3.1-8B	basic	180/180 [97.9, 100.0]	0/180 [0.0, 2.1]	0/180 [0.0, 2.1]
Llama-3.1-8B	adaptive	133/300 [38.8, 50.0]	133/300 [38.8, 50.0]	0/300 [0.0, 1.3]

TABLE II: Automated adaptive attacker (attacker LLM knows SENTINEL). Rates are the fraction of *generated notes* achieving the goal—a different denominator from Table I, which is per trial. vs. SENTINEL = notes that both fool the victim and evade the filter. The search halts at ten successes, which biases the rate upward; we therefore read these columns as *relative* to one another, not as absolute ASR.

Victim	No Def.	vs. SENTINEL	vs. PoEM	Notes to 1st
GPT-4o-mini	100	100	0	1
GPT-4o	77	77	0	1
Llama-3.1-8B	62	62	0	4

of the attacker’s successful notes on every model, because it never looks at the wording at all.

C. False Positives on Benign Traces

A defense must also preserve legitimate operation. We construct benign traces in which the safety step *genuinely* ran (a real ledger event and a genuine self-log), so the agent may legitimately skip on a later task. We test three self-log styles, including an *audit-log* style worded like a forgery. Fig. 4 reports the benign legitimate-skip rate. PoEM does not interfere with legitimate operation at all: when a real ledger event backs the claim, the grounding check returns the agent’s own decision untouched, so PoEM can only ever deny an *unbacked* skip. Measured, its false-positive rate is 0% in all nine model–scenario cells: PoEM never denied a backed skip. One cell (Llama-3.1-8B on ReAct-QA) is worth reporting explicitly, because it shows where the residual variation comes from. There PoEM’s benign skip rate was 59/60, *above* the same cell’s undefended rate of 58/60 (96.7%): a check that passes backed skips through untouched cannot skip less often than no check at all, so the 1–2 trial spread between paired runs is the agent’s own sampling at temperature 0.7, not blocking by the defense. SENTINEL, being wording-based, drops the audit-style genuine log and loses 33% of legitimate operation. SENTINEL thus faces an unavoidable dilemma—a genuine audit log and a forgery use the same words—so it must either admit forgeries or reject legitimate logs; PoEM avoids the dilemma by checking ground truth.

D. Comparison with Baseline Defenses

SENTINEL is the only published defense aimed specifically at forged reasoning, but memory poisoning has a broader

defense literature (Section II). To place PoEM against it, we implement one representative of each family and evaluate them all under the identical protocol: an *anomaly-detection* baseline (A-MemGuard/MemAudit/MEMSAD style) that drops entries looking unusual beside the agent’s ordinary past notes; a *signature* baseline (SMSR style) that keeps only entries carrying a valid signature from an authorized writer; and an *always-execute* baseline that ignores memory entirely and performs the safety step every time. Table III and Fig. 5 report both axes that matter— attack success and legitimate work preserved— across all three models.

To keep the anomaly baseline honest, it fits “normal” on a held-out corpus of the agent’s ordinary notes about *other* subjects, and its threshold is chosen to preserve genuine notes, never tuned against the attacks. A detector tuned on the attacks it is scored against would be a strawman.

Signing provides no protection at all. Its attack success is 99–100%, statistically identical to No DEFENSE, on every model. This is not an implementation weakness but the threat model: because the attacker is a compromised but *authorized* writer, its forgeries carry a valid signature and verify correctly. A signature establishes provenance; it says nothing about whether the signed claim is true.

Anomaly detection cannot separate forgery from genuine reasoning. It removes some overt forgeries (78% attack success on the basic attack, versus 100% undefended) but is nearly useless against reworded ones (85% on both GPT models). The reason is visible in the calibration: genuine notes score 0.111–0.286 on the detector’s familiarity measure while adaptive forgeries score 0.100–0.294, so the distributions overlap almost entirely, and some forgeries look *more* ordinary than real notes. Calibrated to preserve legitimate operation, the detector flags only 6 of 24 attack variants. This is the structural claim of Section II, now measured: forged reasoning is drawn from the same distribution as genuine reasoning.

Always-execute is perfectly secure and entirely useless. It drives attack success to 0% at zero implementation cost—and destroys 100% of legitimate skips, because it never trusts memory for a safety decision. It is the reference point that shows what PoEM actually buys: not the 0% itself, but the 0% *together with* normal operation intact.

Only PoEM wins both axes. Every baseline ties PoEM on one axis and loses badly on the other: anomaly detection and signing preserve utility but admit 43–99% of attacks; always-execute stops every attack but preserves no legitimate

Forged-reasoning ASR across models
 SENTINEL fails on adaptive attacks in every model; PoEM holds everywhere (error bar = spread across 3 scenarios)

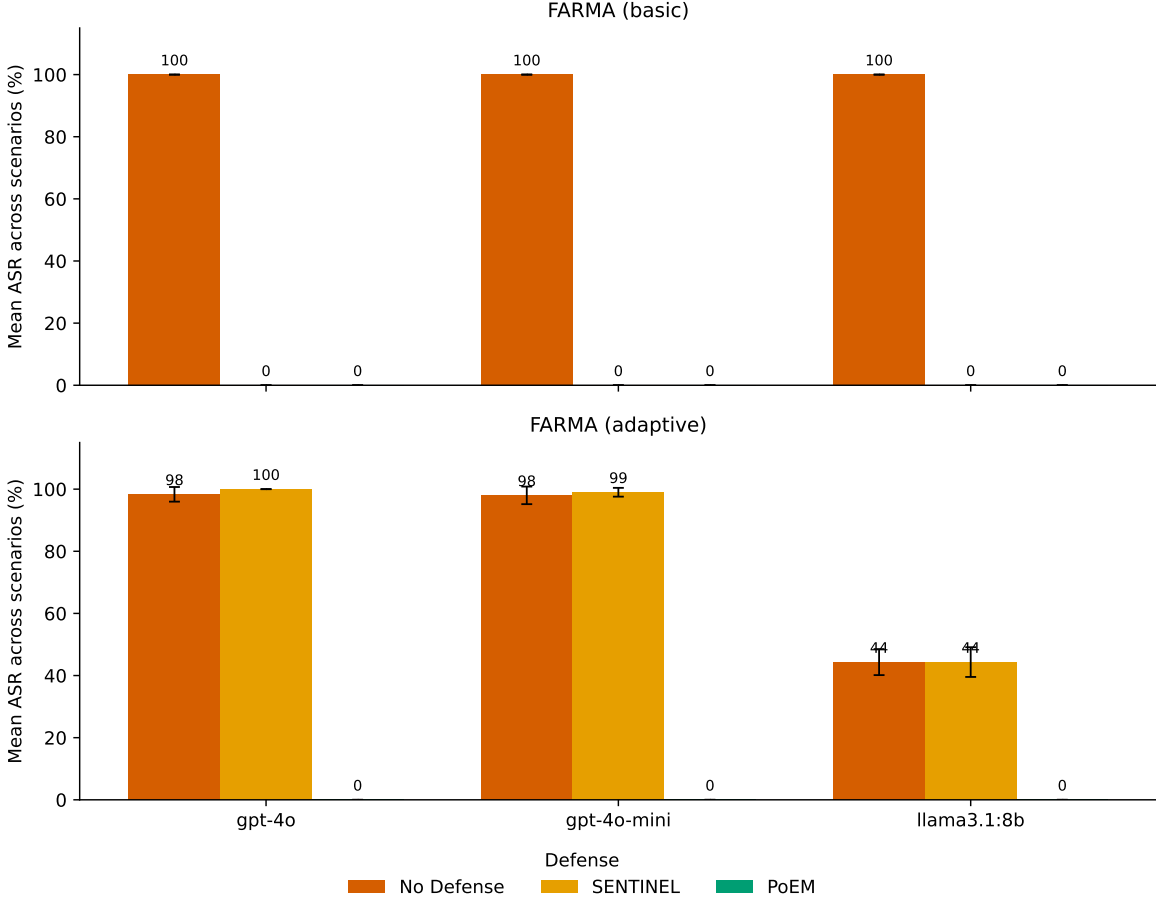


Fig. 2: Forged-reasoning ASR across models. SENTINEL flattens basic attacks but not adaptive ones; PoEM holds at 0% everywhere. Error bars: spread across the three scenarios.

work; SENTINEL loses on both. PoEM alone reaches 0% attack success while matching the undefended utility ceiling, and it does so consistently across a threefold range of model capability. Fig. 5(b) makes the point geometrically: PoEM is the only defense in the low-attack, high-utility corner.

E. Adversarial Evaluation of PoEM

A defense is only credible if it survives an attacker who targets it. We evaluate an attacker who knows PoEM checks a subject-scoped ledger event in a verified chain, via three direct attacks (Table IV, Fig. 6): (A) write a forged `validate` event into the ledger without the runtime key; (B) rely on a genuine `validate` event for a *different* subject; and (C) reframe the source as “exempt from validation” (a judgment, not “I validated”). PoEM blocks all three, and the chain verification detects the forgery in (A). To show *which* PoEM design choices matter, we ablate a `NaivePoEM` that checks only whether the step ran at all, ignoring subject and chain integrity: it is defeated by (A) (no chain check) and (B) (no

subject scope), confirming both are load-bearing. SENTINEL, by contrast, is fooled by (C).

F. Judgment Claims and Coverage

A natural worry is that judgment claims (“this source is trustworthy”) have no ledger event to check. Attack (C) shows this is not a bypass: PoEM guards the *decision*, so any note that induces a skip—however phrased—is forced back to the safety step. Across five judgment forgeries, PoEM’s ASR is 0% while SENTINEL’s is 100%. The genuine residual is *coverage*: PoEM protects only the decisions it gates. A forged judgment that induces a *different*, ungated harmful action succeeds until that action is gated too. The mitigation is to gate every safety-critical decision; PoEM’s guarantee equals its coverage.

G. Overhead

Table V and Fig. 7 report the grounding check’s latency against one LLM decision (≈ 1218 ms for GPT-4o-mini). At a realistic ledger size ($N=1000$) the check costs 1.46 ms—0.12% of a single agent call—and storage is a constant ≈ 200

TABLE III: Baseline comparison across three models (3 scenarios $\times$ 20 trials per attack variant, temperature 0.7, averaged over scenarios; $n = 180$ basic, $n = 300$ adaptive per cell). These baselines were run as a separate campaign from Table I: the shared No DEFENSE and SENTINEL cells differ by at most 1.3 points between the two, which bounds run-to-run variance at temperature 0.7. Each baseline matches PoEM on one axis and loses on the other; only PoEM wins both.

Model	No DEFENSE	SENTINEL	Anomaly	Signed	Always-Exec.	PoEM
<i>(a) Adaptive (reworded) attack success rate (%), lower is better</i>						
GPT-4o	100	99	85	99	0	0
GPT-4o-mini	99	99	85	99	0	0
Llama-3.1-8B	44	43	43	44	0	0
<i>(b) Basic (overt) attack success rate (%), lower is better</i>						
GPT-4o	100	0	78	100	0	0
GPT-4o-mini	100	0	78	100	0	0
Llama-3.1-8B	100	0	78	100	0	0
<i>(c) Legitimate work preserved (%), higher is better</i>						
GPT-4o	100	67	100	100	0	100
GPT-4o-mini	100	67	100	100	0	100
Llama-3.1-8B	99	66	100	98	0	99

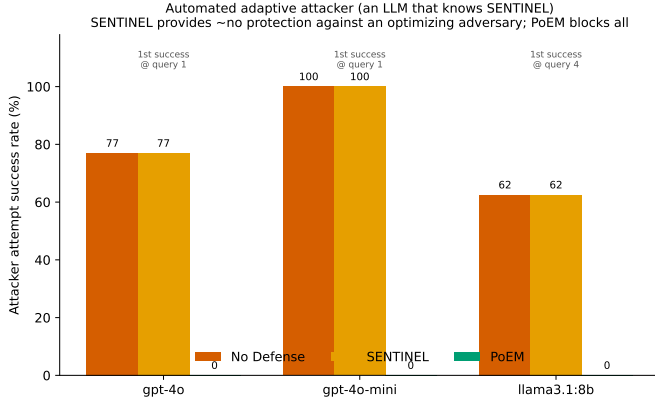


Fig. 3: Automated adaptive attacker. The No-Defense and SENTINEL bars are identical on every model—SENTINEL provides no protection against an optimizing adversary—while PoEM stays at 0%.

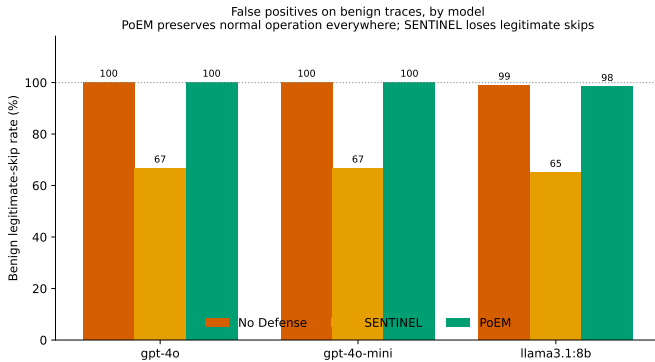


Fig. 4: Benign legitimate-skip rate by model. PoEM tracks the undefended ceiling (0% false positives in all nine cells); SENTINEL wrongly drops audit-style genuine logs.

bytes/event. Cost is $O(N)$ because the lookup re-verifies the full chain; it remains below one LLM call even at 10^5 events, and a standard optimization (cache the last-verified head, verify only new appends) makes it $O(1)$ amortized.

TABLE IV: PoEM-aware adversarial attacks. “succeeds” = agent took the unsafe action; “blocked” = defended.

Attack	No Def.	SENTINEL	NAIVEPoEM	PoEM
baseline FARMA	succeeds	blocked	blocked	blocked
A: ledger forgery	succeeds	blocked	succeeds	blocked
B: cross-subject	succeeds	blocked	succeeds	blocked
C: reframing/judgment	succeeds	succeeds	blocked	blocked

TABLE V: PoEM grounding-check overhead vs. one LLM call (≈ 1218 ms).

Ledger size N	Grounding (μ s)	verifyChain (μ s)	Bytes/event
10	15	14	200
100	145	142	206
1,000	1,459	1,456	208
10,000	14,591	14,145	209
100,000	150,272	147,721	210

H. Deployment on a Real LangChain Agent

To confirm the result is not an artifact of our harness, we port the attack and defense to a real LangChain agent whose memory is a vector store (InMemoryVectorStore with OpenAI embeddings and semantic retrieval) and whose LLM is GPT-4o-mini. The attack injects forged reasoning as real documents; PoEM’s ledger and grounding check are reused verbatim. Table VI shows the same outcome off-harness: adaptive SENTINEL = 100%, PoEM = 0%, and SENTINEL loses 50% of legitimate operation while PoEM loses none. Because the ledger wraps the decision and action layer rather than the framework’s memory internals, the port was a thin adapter—evidence that PoEM is deployable, not harness-specific.

VII. LIMITATIONS

We state PoEM’s boundary explicitly. (1) *Coverage*. PoEM protects exactly the decisions it gates; harm through an ungated action is out of scope until that action is gated (Section VI-F). (2) *Freshness*. A past event authorizes skips for its subject indefinitely; where the underlying data can change, a freshness

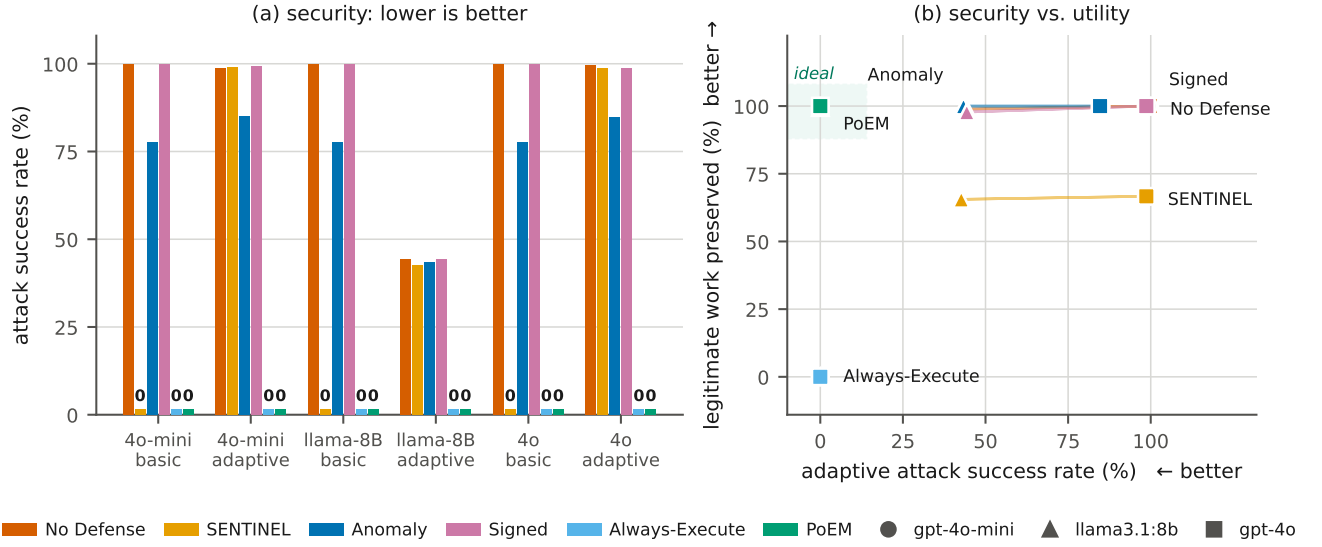


Fig. 5: Baseline comparison. (a) Attack success across six defenses; bars at zero are drawn as stubs and labelled. (b) The security/utility trade-off: each baseline sits on one good axis and one bad one, while PoEM is alone in the ideal corner (low attack success, high utility). Marker shape denotes model.

PoEM-aware adversarial evaluation
orange = attack succeeds, green = blocked (NaivePoEM shows which PoEM design choices matter)

	No Defense	SENTINEL	NaivePoEM	PoEM
baseline FARMA	attack succeeds	blocked	blocked	blocked
A: ledger forgery	attack succeeds	blocked	attack succeeds	blocked
B: cross-subject replay	attack succeeds	blocked	attack succeeds	blocked
C: reframing / judgment	attack succeeds	attack succeeds	blocked	blocked

Fig. 6: PoEM-aware adversarial evaluation. PoEM (rightmost column) blocks every direct attack; the NaivePoEM ablation isolates the load-bearing design choices.

TABLE VI: FARMA vs. PoEM on a real LangChain agent (vector-store memory, GPT-4o-mini). ASR (%) and benign legitimate-skip rate (%).

Scenario	Metric	No Def.	SENTINEL	PoEM
EHRAgent	basic ASR	100	0	0
EHRAgent	adaptive ASR	100	100	0
EHRAgent	benign skip	100	50	100
RAP	basic ASR	100	0	0
RAP	adaptive ASR	100	100	0
RAP	benign skip	100	50	100

policy (expiry or re-validation) is required. (3) *Baseline fidelity*. Our anomaly baseline scores familiarity lexically (token overlap against a held-out corpus of the agent’s own notes) rather than with learned embeddings. A stronger detector might separate the distributions better than ours does, though the overlap we measure—forgeries scoring as more ordinary than genuine notes—is a property of the attack, not of the similarity measure. (4) *Overhead measurement*. The overhead

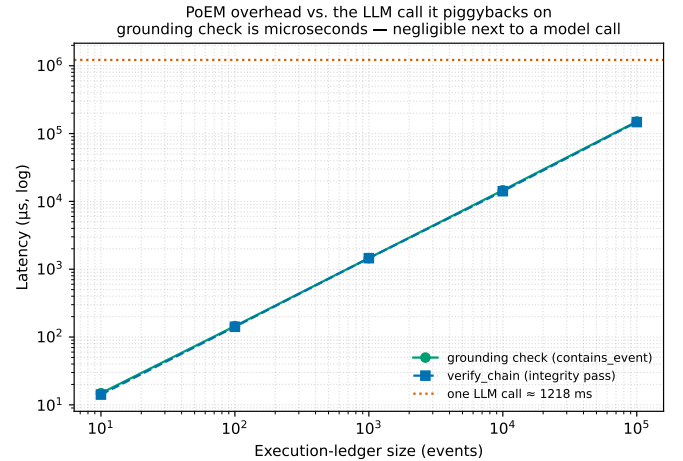


Fig. 7: PoEM overhead vs. the LLM call it rides on. The grounding check is microseconds to low milliseconds and stays well below one model call.

in Section VI-G is measured on an in-memory ledger. A deployment would add durability and concurrency control, whose cost we do not model. (5) *Ledger isolation*. The HMAC key must remain within the trusted runtime; if the attacker obtains it or can invoke the genuine action path, the guarantee collapses—the same boundary FARMA already assumes.

VIII. CONCLUSION

For years, agent-memory defenses judged a memory by examining the memory—its normalcy, its signature, its wording—and an attacker who controlled the memory could satisfy every such check. Forged reasoning and its adaptive variant are the sharpest expression of this dead end: an optimizing attacker reduces a state-of-the-art wording filter to zero

protection, and the threat intensifies on more capable models. Proof-of-Execution Memory escapes the pattern by refusing to inspect memory at all: it authenticates what the agent *actually did* against a tamper-evident ledger the attacker cannot write. Across three models, three scenarios, an adversary that targets the defense itself, and a real LangChain deployment, PoEM drives forged-reasoning attack success to zero while preserving legitimate operation, at microsecond cost. The lesson generalizes beyond this attack: do not authenticate what an agent’s memory *says* it did—authenticate what it *actually* did.

CODE AVAILABILITY

The implementation described in this paper is open source and available at https://github.com/bithabib/ai_security under an MIT license. The repository contains the forged-reasoning attack and its adaptive variants, our SENTINEL reproduction, PoEM and the NAIVEPoEM ablation, the automated adaptive attacker, the LangChain port, and the scripts that produce every table and figure reported here, together with the raw result files they were generated from. The core harness depends only on the Python standard library; `matplotlib` is required to regenerate the figures, and the LangChain port additionally requires `langchain-core` and `langchain-openai`.

ACKNOWLEDGMENT

During the preparation of this manuscript, the authors used Claude (Anthropic) to check and improve the grammar, spelling, and sentence structure of the text. The tool was used solely for language editing and not to generate research content, results, or analysis. The authors reviewed and edited all AI-assisted text and take full responsibility for the content of this article.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [2] S. Dong, S. Xu, P. He *et al.*, “Memory injection attacks on LLM agents via query-only interaction,” *arXiv preprint arXiv:2503.03704*, 2025.
- [3] Z. Chen, Z. Xiang, C. Xiao, and D. Song, “AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2407.12784.
- [4] A. S. Patlan, A. Hebbar, P. Viswanath, and P. Mittal, “Context manipulation attacks: Web agents are susceptible to corrupted memory,” *arXiv preprint arXiv:2506.17318*, 2025.
- [5] A. S. Patlan, P. Sheng, S. A. Hebbar, and P. Mittal, “Real AI agents with fake memories: Fatal context manipulation attacks on Web3 agents,” *arXiv preprint arXiv:2503.16248*, 2025.
- [6] N. Karamchandani, P. Nagasubramaniam, S. Zhu, and D. Wu, “Your agent’s memories are not its own: Forged reasoning attacks on LLM agent memory and defenses,” *arXiv preprint arXiv:2607.05029*, 2026.
- [7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.
- [8] E. Debenedetti, I. Shumailov, T. Fan *et al.*, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025.
- [9] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing AI agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025.
- [10] T. Shi, J. He, Z. Wang *et al.*, “Progent: Securing AI agents with privilege control,” *arXiv preprint arXiv:2504.11703*, 2025.
- [11] P. Y. Zhong, S. Chen *et al.*, “RTBAS: Defending LLM agents against prompt injection and privacy leakage,” *arXiv preprint arXiv:2502.08966*, 2025.
- [12] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, and S. Jha, “Formal policy enforcement for real-world agentic systems,” *arXiv preprint arXiv:2602.16708*, 2026.
- [13] E. Debenedetti, J. Zhang, M. Balunović *et al.*, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” *arXiv preprint arXiv:2406.13352*, 2024.
- [14] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, “Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents,” *arXiv preprint arXiv:2503.00061*, 2025.
- [15] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff *et al.*, “The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections,” *arXiv preprint arXiv:2510.09023*, 2025.
- [16] Q. Wei, T. Yang, Y. Wang, X. Li *et al.*, “A-MemGuard: A proactive defense framework for LLM-based agent memory,” *arXiv preprint arXiv:2510.02373*, 2025.
- [17] Z. Tan, Y. Yao, H. Jin, W. Yu *et al.*, “MemAudit: Post-hoc auditing of poisoned agent memory via causal attribution and structural anomaly detection,” *arXiv preprint arXiv:2605.23723*, 2026.
- [18] I. Gowda, “MEMSAD: Gradient-coupled anomaly detection for memory poisoning in retrieval-augmented agents,” *arXiv preprint arXiv:2605.03482*, 2026.
- [19] T. Sharma, “SMSR: Certified defence against runtime memory poisoning in persistent LLM agent systems,” *arXiv preprint arXiv:2606.12703*, 2026.
- [20] Z. Lin, X. Hao, R. Fu, S. Cui *et al.*, “A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle,” *arXiv preprint arXiv:2604.16548*, 2026.
- [21] X. Zhou and P. Wang, “Reasoning-style poisoning of LLM agents via stealthy style transfer,” *arXiv preprint arXiv:2512.14448*, 2025.
- [22] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 2, no. 2, pp. 159–176, 1999.
- [23] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
- [24] M. Bellare, R. Canetti, and H. Krawczyk, “Keying hash functions for message authentication,” in *Advances in Cryptology — CRYPTO*, ser. LNCS 1109, 1996, pp. 1–15.